PhiUSIIL Phishing URL Detection: A Comprehensive Statistical Analysis

By: Patrick Muthii

Contact: patrickmuthii97@gmail.com

Abstract

Phishing attacks are now prevalent across almost all sectors of the technology world. Every device that connects to the internet is prone to a cyber threat when fetching information online. Cybercriminals are always exploring nefarious ways to steal private and personal data

from unsuspecting users who browse the internet. The technology boom has made it easier for users to not only connect online but also allowed uncontrolled and unregulated freedom of hosting websites. As a result, there is an increased prevalence of phishing attacks that take advantage of unsuspecting users and thereby create illegitimate websites that mimic legitimate websites with a goal of stealing sensitive data.

This report relies on the PhiUSIIL Phishing URL Dataset, a dataset that contains 134,850 legitimate and 100,945 phishing URLs with 46 distinct features, to develop a machine learning solution for detecting phishing links. It covers the procedural ways involved beginning with data preprocessing, exploratory data analysis, and feature engineering, model training and evaluation. To determine the optimal algorithm for the development of the best machine learning model, different algorithms are tested. Statistical analyses have also been implemented to reveal useful patterns that can distinguish phishing urls from legitimate ones.

Throughout the report, there are key areas covered such as the methodology, findings, recommendations for the stakeholders, for instance the developers, organizations and other industries. The overall aim is to practically show the role of machine learning in fraudulent URL detection.

Group Members

Table of Contents

1. **Introduction……..4**
2. **Literature Review……..5**

3. Dataset Overview……..7

4. Methodology……10

4.1 Data Preprocessing**……..10**

4.2 Feature Selection and Engineering**……..12**

4.3 Evaluating Machine Learning Models…….**13**

4.4 Feature Correlation…….….**14**

4.5 Comparison of Models…….……..…...**14**

4.6 Model Selection…….….**15**

4.7 Training and Validation Strategy…….….**15**

5. Statistical Analysis……16

5.1 Exploratory Data Analysis**……..15**

5.2 Results from EDA…….**……..18**

6. Key Findings……25

6.1 Evaluation metrics and Model Performance**…..……..26**

6.2 Insights from Model Performance……**……...……...26**

6.3 Limitations and Challenges……..……...**27**

7. Recommendations..……27

8. Conclusion……...…….29

9. Bibliography…….30

10. Appendix……..…….…….33

1. Introduction

Due to the growth of internet usage, online transactions and the sharing of sensitive information are common. However, this has given rise to cybercrime, a modern form of crime exploiting technologies. Among the various forms of cybercrime, phishing has emerged as a dominant threat. Phishing is a social engineering attack where attackers deceive users by creating

fake websites that mimic legitimate ones to steal sensitive data such as passwords, personal information, and financial details. These malicious acts are conducted through email, instant messaging, and illegitimate web pages. Phishing through Uniform Resource Locators (URLs) is one of the most common cases (Mohammad et al., 2015b).

URLs, or web addresses, are the key areas of websites on the internet and can be either benign or malicious (Chiew, Yong et al., 2018). Benign URLs lead to legitimate and secure websites, malicious URLs are created by cybercriminals to impersonate trusted websites. These malicious URLs are often distributed through advertisements, emails, or embedded in other websites. Once a user interacts with a malicious URL, they may unknowingly or unwillingly disclose sensitive information or download harmful software. It is challenging for users to distinguish between them because of the similarity between malicious and benign URLs. Therefore, there is a need for intelligent detection systems.

Phishing attacks have become prevalent as reported by the Anti-Phishing Working Group (APWG), where over 600,000 unique phishing websites were detected in the last quarter of 2020 alone (Anti-Phishing Working Group, 2021). Financial institutions were the primary targets (24.9%), followed by social media platforms (23.6%), Software as a Service (SAAS) providers, and webmail services (19.6%). Phishing often leads to economic losses which affect both corporations and individuals. To date, corporations have lost over $411 million (Bose & Leung, 2014).

To fight this threat, researchers have developed various machine learning techniques to detect and prevent phishing attacks. These techniques rely on datasets containing features extracted from URLs to distinguish between malicious and benign links. This report focuses on leveraging the PhiUSIIL Phishing URL Dataset, which consists of 134,850 legitimate URLs and

100,945 phishing URLs with 46 distinct features. This dataset can be used to develop a machine learning solution to detect phishing URLs effectively.

This report outlines a process of building a phishing link detector, starting with data preprocessing, exploratory data analysis, and feature engineering. It also evaluates several machine learning algorithms such as the Random Forest, Decision Tree, and Logistic Regression models to identify the most effective model for detecting phishing URLs. Statistical analyses are also conducted to uncover patterns and insights that differentiate phishing URLs from legitimate ones.

Lastly, this report presents findings, methodologies, and recommendations to show the critical role of machine learning in addressing phishing threats. The overall goal is to provide a practical framework for detecting malicious URLs and to safeguard users and organizations from phishing attacks.

2. Literature Review

Phishing attacks have evolved over the years, and currently targets unsuspecting users across various online platforms. Phishing methods used by cybercriminals have changed too hence it is a persistent and growing threat. Studies show that phishing campaigns last an average of 21 hours (Oest et al., 2020), which means real-time detection mechanisms need to be adopted. Unfortunately, most existing defenses, such as browser-based tools like Google SafeBrowsing and PhishTank, rely on blocklists. These systems only protect users from previously reported phishing websites but not on new unreported malicious URLs. As a result, users often suffer phishing attacks before these websites are flagged, and possibly expose sensitive credentials to attackers.

There is an alarming societal and economic impact of phishing attacks. Serious financial losses have been reported in fields such as finance, social media platforms, and SaaS providers. According to the Anti-Phishing Working Group (2021), 83% of phishing websites now use the Secure Sockets Layer (SSL) protocol, which displays the green padlock on browsers to deceive users that the sites are secure. Hence, these traditional detection features, such as HTTPS presence, are less effective. Adebowale et al. (2019) and Li et al. (2019) emphasized how the reliance of earlier models on HTTPS was a distinguishing feature, but it can no longer be trusted.

The effectiveness of machine learning models heavily depends on the quality and relevance of the data used during training. Sanchez-Paniagua et al. (2022) demonstrates the problem of outdated datasets which fail to discover ever-changing phishing techniques, which ultimately result in poor performance of trained models. Phishing URLs are short-lived, hence they should be captured immediately in order to develop robust datasets. However, efforts to build offline phishing datasets have been limited. Chiew et al. (2018) made an attempt to create such a dataset but encountered issues with legitimate URLs and screenshots, which affected its reliability.

The issue of bias in dataset construction poses a major concern where the focus is on index pages, ignoring real-world scenarios where phishing websites often use login forms to steal credentials. El Aassal et al. (2020) addressed this bias by incorporating login pages into their datasets, but the proportion of login forms between phishing and legitimate classes was rendered imbalanced. This discrepancy invalidates machine learning models, reducing their effectiveness in detecting phishing URLs. To address this, newer datasets have been developed, such as those that balance the number of login forms in both classes.

Machine learning is a powerful tool in detecting phishing attacks. Through extracting features from URLs, such as domain age, IP address usage, and URL length, machine learning models can differentiate between malicious and benign links. They provide an advantage over traditional blocklist-based systems as they can identify phishing URLs before they are reported. On the other hand, the increasing complexity of phishing websites - as they too use SSL, demands more advanced feature extraction and model refinement, for example, specific statistical analyses and accurate feature engineering so as to improve the model performance.

Phishing is a pervasive threat with urgent implications. Researchers need to address the challenges of outdated datasets, misleading indicators like SSL, and dataset biases and even explore new innovative approaches to phishing detection. Machine learning presents a palatable solution, but its effectiveness depends on how it addresses these challenges, probably using robust datasets and advanced techniques. This study builds on existing research by leveraging a robust dataset to develop a machine learning solution capable of detecting phishing URLs in real-world scenarios.

3. Dataset Overview

This report relies on PhiUSIIL dataset, a substantial dataset comprising 134,850 legitimate and 100,945 phishing URLs. It contains an analysis of the most recent URLs and its 46 Features are extracted from the source. Its focus is on developing a diverse security profile that fulfills diverse user requirements. The PhiUSIIL phishing URL dataset can be used to train the machine learning model. We will start by explaining the PhiUSIIL phishing URL dataset features and then discuss the methodology in the following section.

In this section, we provide an overview of the dataset, followed by a discussion of the methodology in the following section. The PhiUSIIL phishing URL dataset was constructed

using legitimate URLs sourced from the Open PageRank Initiative Anon (2023g) while the phishing URLs are fetched from PhishTank, OpenPhish, and MalwareWorld. Some features contained in the dataset to help in detecting phishing URLs are also described below.

Feature	**Description**
HasFavicon	Legitimate websites have their website logo included in the favicon tag while illegitimate might lack
IsResponsive	Many phishing websites are not designed to be responsive across different devices due to their quick, poorly optimized development.
NoOfURLRedirect	Phishing websites often use redirects, such as JavaScript or meta tags, to mislead users and direct them to unexpected pages.
HasDescription	Legitimate websites typically provide detailed page descriptions using the 'description' meta tag, which is often absent in phishing websites.
NoOfPopup, NoOfiFrame	Phishing websites may use pop-ups or iframes to distract users and capture sensitive data. These can be detected by examining tags such as window.open and iframe in the HTML.

HasExternal FormSubmit	Forms that submit data to external URLs are a common indicator of phishing attempts.
HasCopyrightInfo, HasSocialNet	Legitimate websites generally include copyright and social media information, which phishing sites often omit.
HasPasswordField, HasSubmitButton	The presence of password and submit fields can indicate phishing attempts, especially if found alongside other suspicious elements.
HasHiddenFields	Hidden fields in the HTML code can be used by phishing sites to capture sensitive data without user awareness.
Bank, Pay, Crypto	The use of terms like "bank," "pay," or "crypto" often signals attempts to extract financial information fraudulently.

Table 1. Some Features of the PhiUSIIL phishing URL dataset

Other features include the use of screenshots of legitimate websites to appear authentic. A disproportionate number of images on smaller pages can indicate a phishing activity and excessive JavaScript usage, such as for pop-ups or misleading elements, can indicate a phishing website and also, hyperlinks in phishing websites frequently to mislead users by appearing legitimate while redirecting to phishing pages. A large number of self-referencing, empty, or external links can raise suspicion.

This dataset's extensive nature is useful in capturing a diverse form of phishing urls and hence effective to train a machine learning model.

```
[5]: df.head()
```

[5]:

harContinuationRate	TLDLegitimateProb	...	Pay	Crypto	HasCopyrightInfo	NoOfImage	NoOfCSS	NoOfJS	NoOfSelfRef	NoOfEmptyRef	NoOfExternalRef	label
1.000000	0.522907	...	0	0	1	34	20	28	119	0	124	1
0.666667	0.032650	...	0	0	1	50	9	8	39	0	217	1
0.866667	0.028555	...	0	0	1	10	2	7	42	2	5	1
1.000000	0.522907	...	1	1	1	3	27	15	22	1	31	1
1.000000	0.079963	...	1	0	1	244	15	34	72	1	85	1

```
[6]: print("A. Basic statistical summaries")
print("\nDataset shape:", df.shape)
print("\nColumn types:")
print(df.dtypes)
print("\nSummary statistics for numerical columns:")
print(df.describe())
```

```
A. Basic statistical summaries

Dataset shape: (235795, 56)

Column types:
FILENAME                object
URL                     object
URLLength                int64
Domain                  object
DomainLength             int64
IsDomainIP               int64
TLD                     object
URLSimilarityIndex     float64
```

Figure 1. A summary statistics of the PhiUSIIL dataset

4. Methodology

4.1 Data Preprocessing

Data preprocessing is a critical step to ensure the dataset used for training and evaluating machine learning models is accurate, reliable, and suitable for the intended task (Kotsiantiset al., 2006). High-quality preprocessing helps create trustworthy models that produce effective predictions. We performed the following steps were to validate and prepare the dataset:

URL Verification: All URLs included in the dataset were sourced from valid and credible platforms

Handling Null Values: No null values were found. This ensures data completeness. Addressing Missing Values: The dataset contains no missing entries. Duplicate Records: All records in the dataset were verified to be unique. Zero Variance Features: Each feature in the dataset was analyzed, and none were found to have identical data values throughout. Infinite Values: The dataset was reviewed to ensure no positive or negative infinite values were present. Class Balance: The dataset consists of 57% legitimate URLs and 43% phishing URLs.

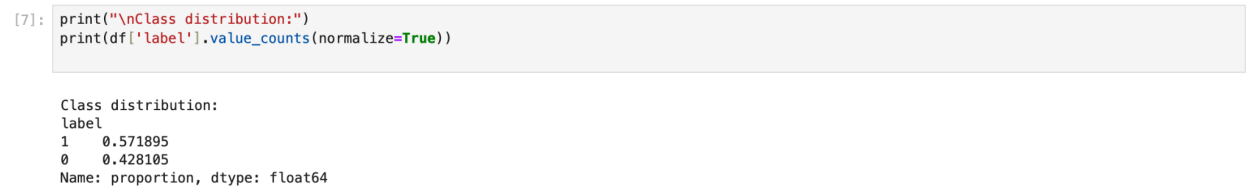


Figure 2. Feature percentage distribution

Validating the dataset through machine learning algorithms helps to assess the dataset's quality and establish baseline performance metrics. This process ensures that the dataset is suitable for training models and provides a benchmark for future model improvements. A dataset with errors or biases can lead to models that propagate these issues. Key validation steps include:

Running Baseline Models: Testing the dataset on a variety of machine learning algorithms helps identify potential flaws and ensures the dataset's reliability (Kotsiantis et al., 2006).

Error Propagation Mitigation: Early identification of errors or biases in the dataset prevents their transfer into the model, leading to more accurate predictions. These preprocessing steps ensure the dataset is clean, balanced, and ready for effective use in phishing URL detection.

4.2 Feature Selection and Engineering

The performance of a machine learning model depends on its ability to identify and use the most relevant features in the dataset for classification and prediction (Chen et al., 2020). Effective feature selection and engineering enable the model to better understand patterns and relationships within the data. Therefore, efficient feature selection leads to classification accuracy. This section discusses the evaluation of machine learning algorithms and the importance of feature importance analysis.

4.2.1 Importance of Feature Selection

In datasets with multiple features, evaluating feature importance helps to understand the attributes that contribute most to the target variable's prediction because features in a dataset hold different significance; some may be redundant or irrelevant, or even cause noise that degrades model performance (Hall, 1999). Feature selection avoids this by focusing on the most impactful features, which leads to not only faster and efficient model computation but also improved model interpretability.

We assessed feature importance using models like Random Forest, which supports built-in mechanisms to rank features based on their contribution to the final prediction. The analysis revealed that URLSimilarityIndex was the most significant indicator of a phishing URL.

Some advantages associated with feature importance analysis are improved accuracy as the model can focus on the most predictive indicators, leading to better classification results, minimal overfitting which ensures the model generalizes well to unseen data and also interpretability since the features guide the model on the predictions which is crucial for trust in real-world applications.

4.3 Evaluating Machine Learning Models

To identify the most effective algorithm for phishing URL detection, we evaluated several machine learning models, including Random Forest, Decision Tree, and Logistic Regression. The selection of the ideal algorithm was based on the algorithm that could handle a dataset with numerous features while providing insights into feature importance and correlation.

4.3.1 Random Forest Classifier:

Ghiasi and Zendehboudi (2021) describe Random Forest as an ensemble learning method that combines multiple decision trees to improve classification accuracy and reduce overfitting. It is effective for datasets with many features because it evaluates feature importance based on the reduction in impurity across all trees in the ensemble. In this study, Random Forest provided a detailed ranking of feature importance which helped to identify key indicators such as URLSimilarityIndex, HTTPS, and others. Its powerful performance and ability to capture non-linear relationships makes it ideal for scenarios with complex feature interactions like our dataset.

4.3.2 Decision Tree Classifier:

Decision Tree models are interpretable and versatile as they can handle both numerical and categorical data. Decision Trees can sometimes overfit the data, especially when the dataset has many features. Nonetheless, it has the ability to visualize decision paths and is useful for feature analysis and initial exploration.

4.3.3 Logistic Regression:

Logistic Regression is a linear model that works well with smaller feature sets and when the relationships between features and the target variable are mostly linear. It is less effective than Random Forest for datasets with complex, non-linear feature interactions; but can serve as a baseline model for comparison. Logistic Regression can also help to understand the importance of individual features in binary classification tasks (Saarela & Jauhiainen, 2021).

4.4 Feature Correlation

Analyzing feature correlation helps identify relationships between variables that may influence the model's performance. Strongly correlated features can introduce redundancy, which can negatively affect the model's ability to generalize (Toloşi & Lengauer, 2011). For instance, in this study, there was evidence of a correlation between HTTPS and the likelihood of a URL being legitimate. One way to solve this is to examine the pairwise correlations and by removing highly correlated features (for example, those with a correlation coefficient greater than 0.9). This way, we ensure that the model is not biased by overlapping information. This step is important when dealing with datasets with many features, as it reduces multicollinearity and enhances the interpretability of feature importance scores.

4.5 Comparison of Models

In our investigation, Random Forest was the most effective, due to its ability to handle large feature sets and capture non-linear relationships. Its ensemble nature avoided overfitting compared to the Decision Tree model. Logistic Regression failed to achieve similar accuracy due to the dataset's complexity and the non-linear interactions among features. The combination of feature

importance analysis and correlation studies allowed us to refine the dataset and select the most informative features

4.6 Model Selection

In phishing detection, the choice of an appropriate model can guarantee a high accuracy and reliability. The Random Forest classifier emerged as the most effective model based on its robust performance across key metrics such as Accuracy, Precision, Recall, and F1-Score during the evaluation phase. The choice was also influenced by its ability to handle datasets with multiple features well, with a dependable ensemble of decision trees that averages their predictions, reduced overfitting and improved generalization. The model can capture complex relationships and interactions between features especially in our dataset that contains 46 features. In addition this algorithm provides feature importance scores, which allow us to identify the most influential indicators of phishing URLs.

4.7 Training and Validation Strategy

The training and validation process performed a cross-validation approach to ensure reliable performance evaluation and mitigate overfitting. We split the dataset into training and testing sets, with 80% of the data allocated for training and 20% for testing. We also applied a 5-fold cross-validation for a deeper evaluation where we divided the training data into five subsets, training the model on four subsets, and validating it on the remaining one. The process was repeated five times, with each subset serving as a validation set once. The results were averaged to obtain reliable performance metrics, including Accuracy, Precision, Recall, F1 Score, and ROC AUC. We then created a pipeline for each identifier that included data scaling using StandardScaler to normalize the features, and model training to ensure all algorithms were

evaluated consistently. We set the number of estimators in Random Forest to help in our hyperparameter tuning and ensure overall accuracy and generalizability.

5. Statistical Analysis

5.1 Exploratory Data Analysis (EDA)

Although all the features are derived from the URL or HTML code, it is important to explore their relevance for machine learning. Determining and quantifying the importance of individual features helps select and discard features in machine learning. Having a high number of features can increase the complexity of the model and potentially make it harder to interpret. Feature importance helps identify less influential features that can be safely removed without significantly affecting model performance. Feature importance scores can help identify which features impact the model's predictions most and are relevant for machine learning. We followed the following steps to identify relevant features. Feature importance provides insights into which variables mostly influence model predictions, which helps to optimize performance and interpretability. In the attached visualization, we compare the top 10 features for Random Forest, Decision Tree, and Logistic Regression models to identify consistently important features.

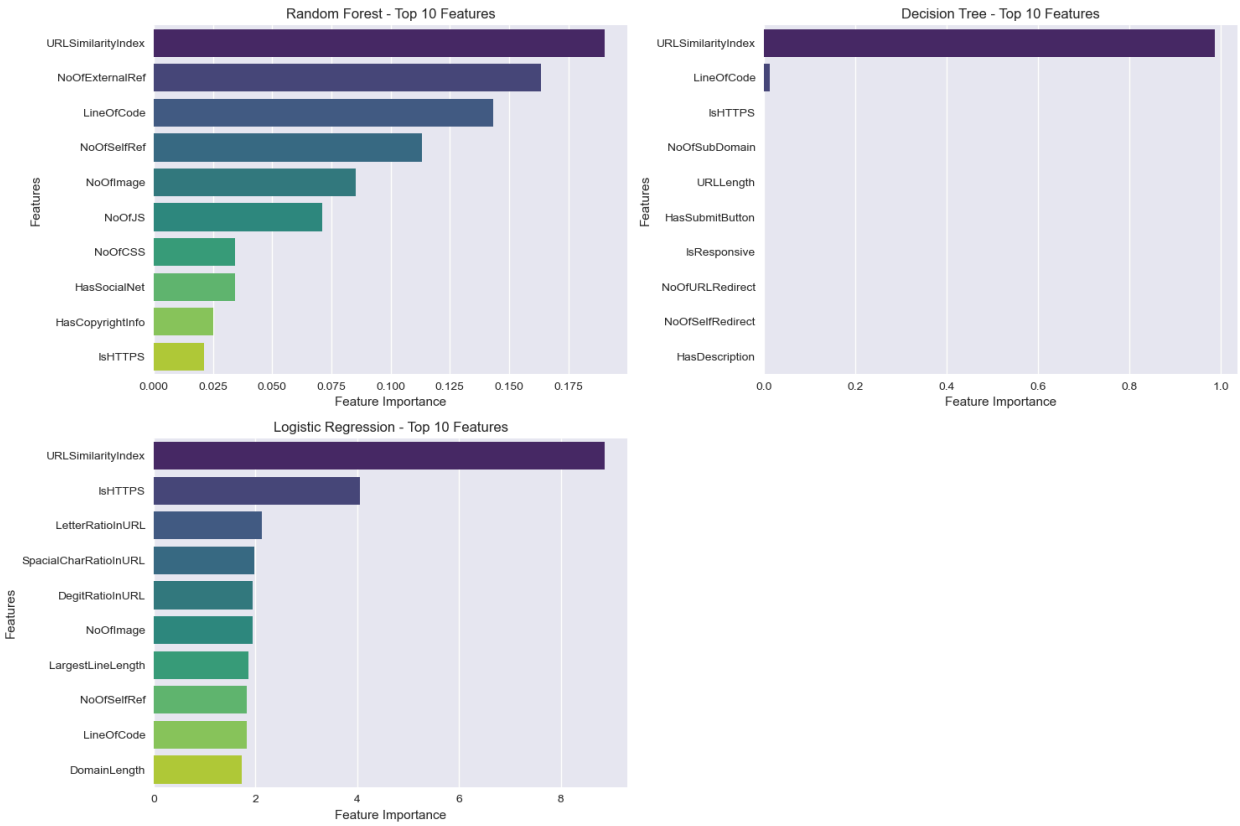


Figure 3. A comparison of the top 10 features for Random Forest, Decision Tree, and Logistic Regression models

5.1.1 Random Forest

The Random Forest model consistently identifies features that contribute the most to classification. From the visualization, the *URLSimilarityIndex* is the most important feature, followed by *NoOfExternalRef*, *LineOfCode*, and *NoOfSelfRef*. These features are related to the structure and behavior of the url, and their scores suggest a strong predictive power. Also since certain features like *HasSocialNet* and *HasCopyrightInfo*also rank high, we can conclude that metadata features are also important.

5.1.2 Decision Tree

Similar to Random Forest, the *URLSimilarityIndex* feature appears to be a strong indicator of phishing links. Since, this algorithm hasn't shown much ensemble in its selection of the features, it suggests that the Decision Tree relies heavily on this singular feature to make decisions. This simplicity can enhance interpretability, but again could disadvantage a machine learning model due to an over-reliance on specific attributes, which may cause reduced generalization.

5.1.3 Logistic Regression

The Logistic Regression model also ranks *URLSimilarityIndex* and *IsHTTPS* as the most important features but returns other influences such as *LetterRatioInURL* and *SpecialCharRatioInURL*. These features measure textual properties, and are an indication of Logistic Regression's tendency to capture linear relationships in data. Features like *NoOfImage* and *LargestLineLength* are due to its ability to utilize simple numerical metrics effectively. In all algorithms *URLSimilarityIndex* ranks as the most influential feature. From our findings, the Random Forest model provides a clearer and broader distribution of important features.

```
Top 10 Most Important Features:
                   Feature          Score
3      URLSimilarityIndex  671857.021357
36           HasSocialNet  376744.060223
43       HasCopyrightInfo  291213.405607
32         HasDescription  214554.534151
21                IsHTTPS  139101.470625
25  DomainTitleMatchScore  122616.776205
37        HasSubmitButton  118640.503918
29           IsResponsive  101521.680953
26     URLTitleMatchScore   96765.322906
20  SpacialCharRatioInURL   93831.344742
```

Figure 4. Top 10 Most Important Features and their scores

5.2 Results from EDA

5.2.1 Key Numerical Features

Below, we conduct an analysis of key numerical features using boxplots and the Mann-Whitney U test to reveal further insights into phishing and legitimate URLs. From the observations of the behavior of these features URLLength, DomainLength, NoOfSubDomain, NoOfLettersInURL, NoOfDegitsInURL, and NoOfOtherSpecialCharsInURL in the graphs, we can conclude that: Phishing URLs are longer than legitimate ones, as seen from the distributions. It is also likely due to the common practice of embedding multiple subdomains or unnecessary characters to obfuscate intent. The illegitimate URLs often have a higher count of subdomains, as they try to mimic legitimate websites by creating complex domain structures. The same is true for the counts of letters, digits, and special characters which are generally higher for phishing URLs. Finally, the statistical tests of the p-values for all features are below 0.05 which are significant differences between phishing and legitimate URLs for these metrics.

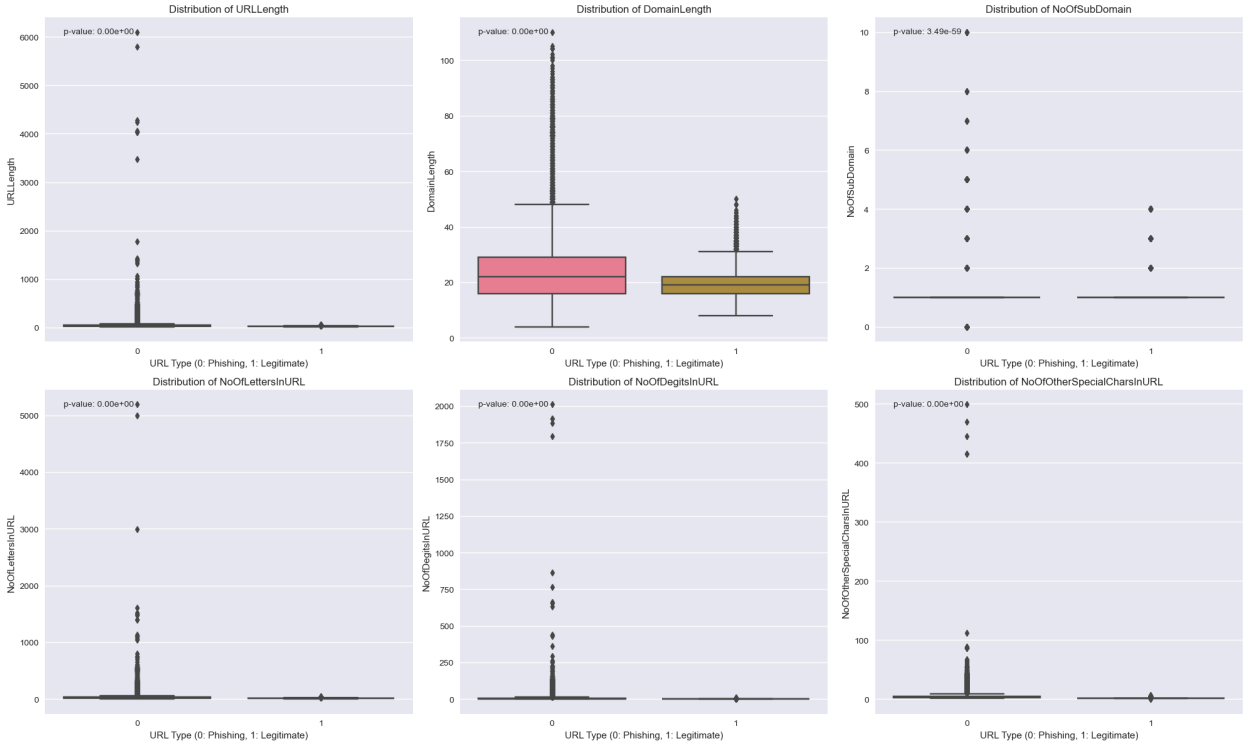


Figure 5. Key Numerical Features visualization

Hence, numerical features, especially URL length and domain complexity, are important when distinguishing between phishing and legitimate URLs.

5.2.2 Analysis of Correlation Matrix

The correlation matrix provides an overview of the relationships between features which can identify the strong predictors for phishing detection. Our analysis covers four key relationships identified in the PhiUSIIL dataset.

Some features like NoOfLettersInURL and NoOfDigitsInURL show high correlation with URLLength which reiterates the finding above that phishing URLs tend to have more characters and digits. Similarly, DomainLength and URLLength are negatively correlated with the label(phishing vs. legitimate), suggesting phishing URLs tend to be more complex. Conversely, IsHTTPS has a positive correlation with legitimate URLs. Features like HasHiddenFields and HasPasswordField show a slight positive relationship with phishing URLs. This means they play an important role in identifying malicious intents. Despite the large number of features, no single feature overwhelmingly determines phishing, but a combination of strongly correlated metrics, such as URL and domain characteristics, is likely effective for detection. This information has been visualized using the heatmap below.

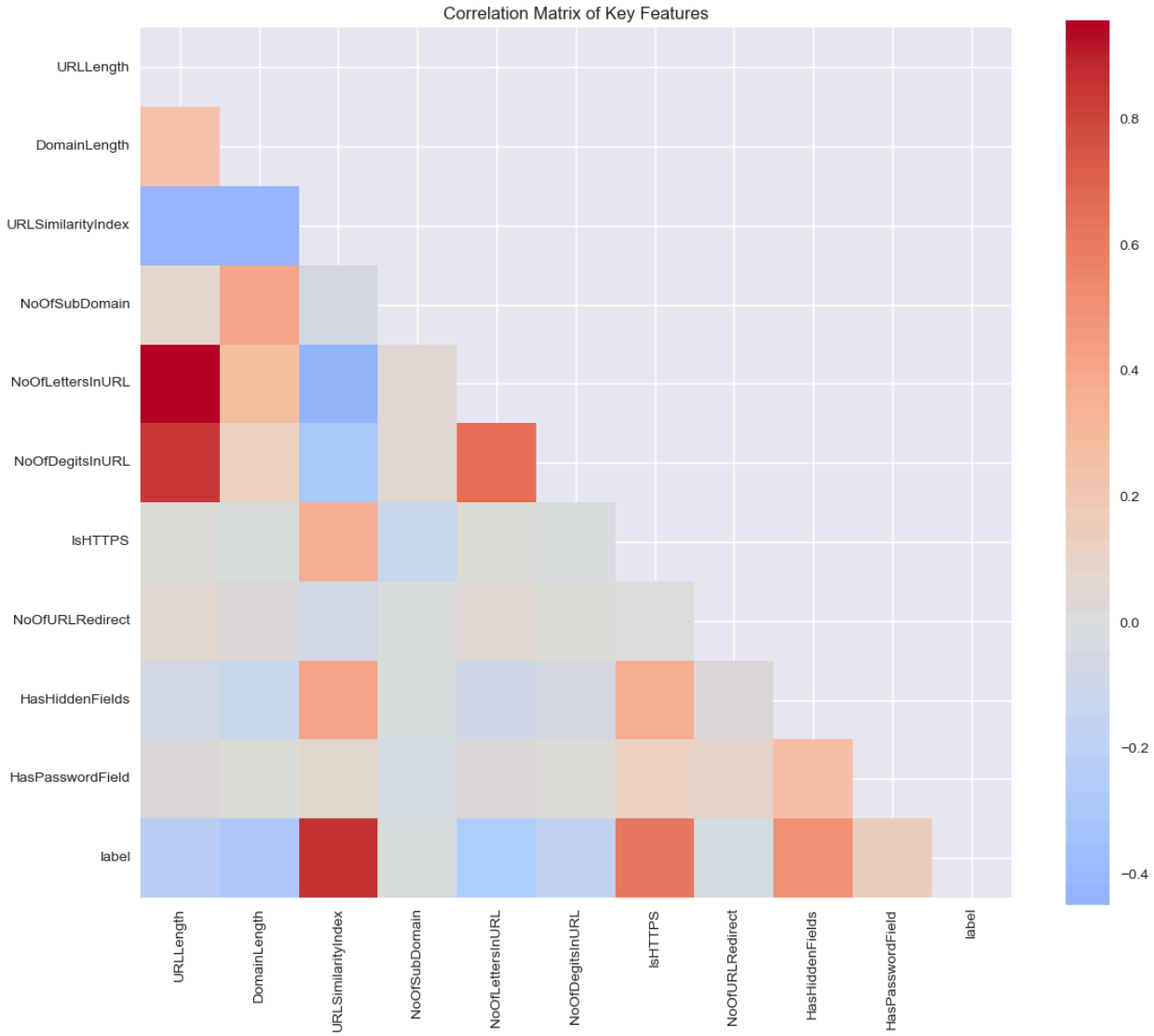


Figure 6. Feature Correlation matrix visualization

5.2.3 Security Indicators Analysis

The Security Indicators Analysis plot shows the distinguishing characteristics of phishing and legitimate URLs across five key security indicators discussed below. The plot shows a stark contrast in HTTPS implementation between phishing and legitimate URLs. Legitimate URLs possess a higher percentage of HTTPS usage, which informs that secure connections are a strong marker of website authenticity. Phishing URLs exhibit markedly lower HTTPS adoption because

secure protocols are less prevalent in malicious websites. There's a big difference in password field characteristics. Legitimate URLs show a balanced distribution of password fields, while phishing URLs display an irregularity as a form of deceptive implementation. The analysis reveals variations in hidden field usage between phishing and legitimate URLs. This could indicate that phishing sites might set hidden fields differently to mask their malicious intent or manipulate user interactions. There are notable differences in external form submission patterns that provide insights into potential red flags for phishing detection. Legitimate URLs appear to have more standardized form submission practices. Also, the presence of social network-related elements shows variations between phishing and legitimate URLs, a likely indication of how malicious sites might attempt to mimic legitimate social media integrations.

The statistical significance (p-values) for each feature indicates that these differences are not random, but represent meaningful distinctions between phishing and legitimate URLs. This finding shows the complexity of phishing detection and the importance of multi-dimensional analysis in creating effective cybersecurity solutions.

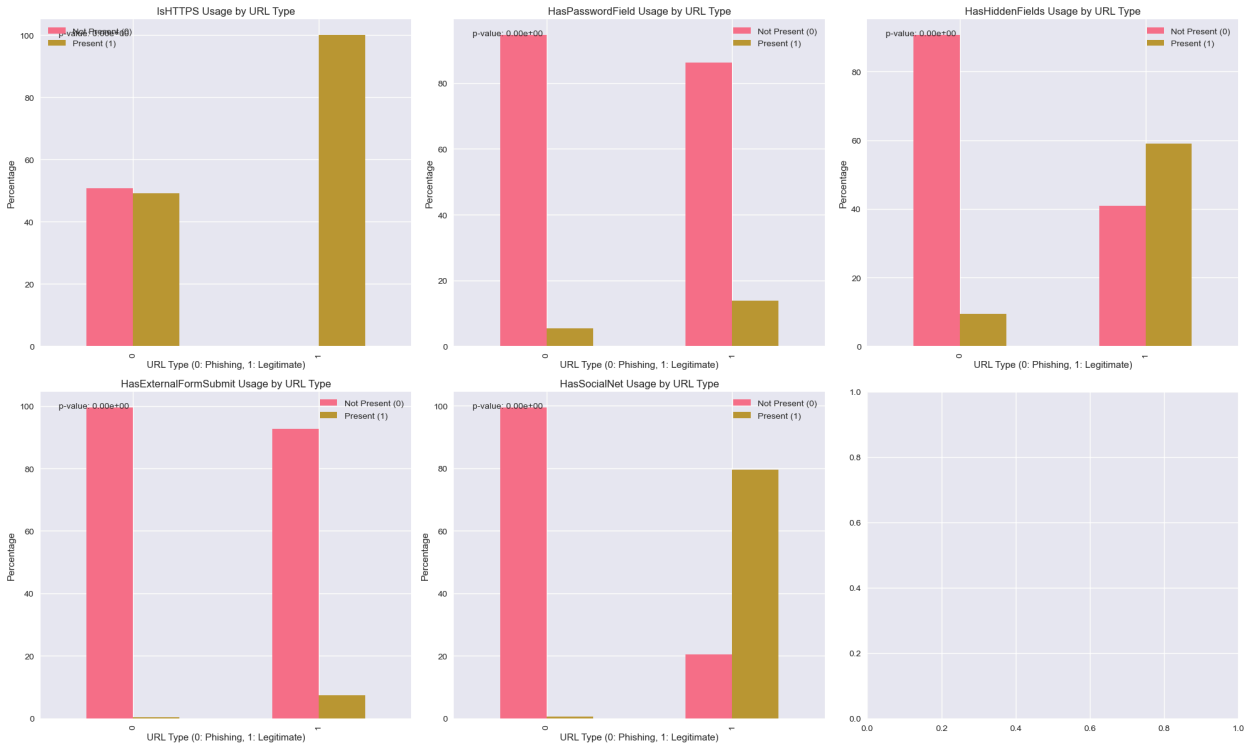


Figure 7. The Security Indicators Analysis plot

5.2.4 Model Performance Metrics

Our Model Performance Comparison compared the performance of three different classifiers: Random Forest, Decision Tree, and Logistic Regression. The performance metrics evaluated include Accuracy, Precision, Recall, F1-Score, and ROC AUC. Additionally, it performs 5-fold cross-validation to assess the models' generalization capabilities. The results show that all three models achieve near-perfect performance, with Accuracy, Precision, Recall, and F1-Score all around 1.0. The ROC AUC scores are also 1.0, which means the models can perfectly distinguish between phishing and legitimate URLs. The cross-validation results return mean scores close to 1.0 and minimal standard deviations. The plot of the Receiver Operating Characteristic (ROC) curves visually demonstrates a brilliant performance of the classifiers. All three models' ROC curves are indistinguishable from the ideal curve, with the area under the

curve (AUC) being 1.0 for each which means the models are highly effective in balancing the trade-off between true positive rate and false positive rate.

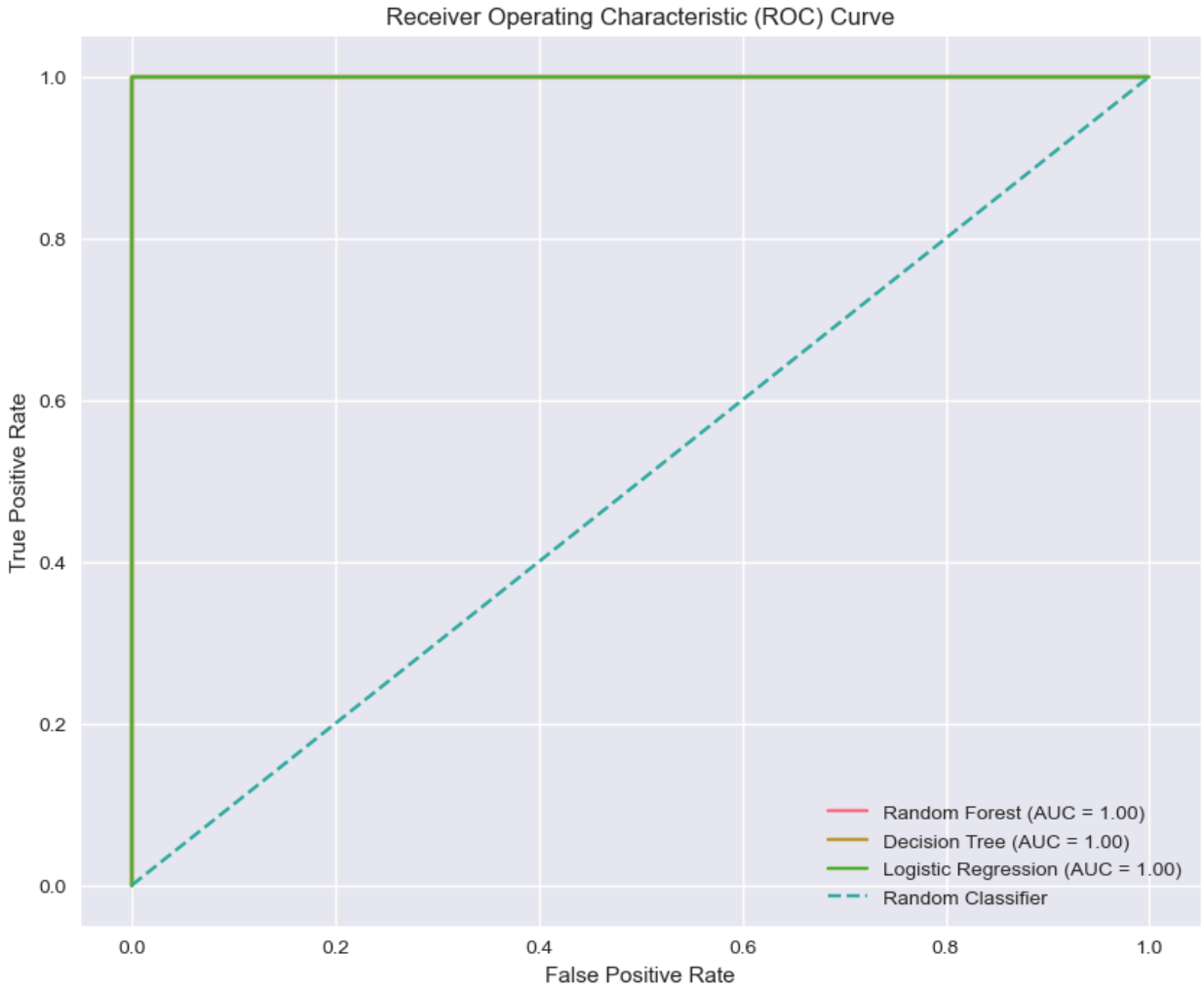


Figure 8. Model Performance Comparison

The Detailed Classification Report provides a comprehensive breakdown of the Random Forest model's performance on the test data. It includes precision, recall, F1-score, and support for both the 'phishing' and 'legitimate' classes. This report shows that the Random Forest model achieves perfect classification, with 100% precision, recall, and F1-score for both classes. Hence, the model correctly identifies all phishing URLs as phishing and all legitimate URLs as

legitimate, without any false positives or false negatives. The confusion matrix, on the other hand, depicts the model's perfect classification. All data points are correctly classified, with the correct classification of both phishing and legitimate urls.

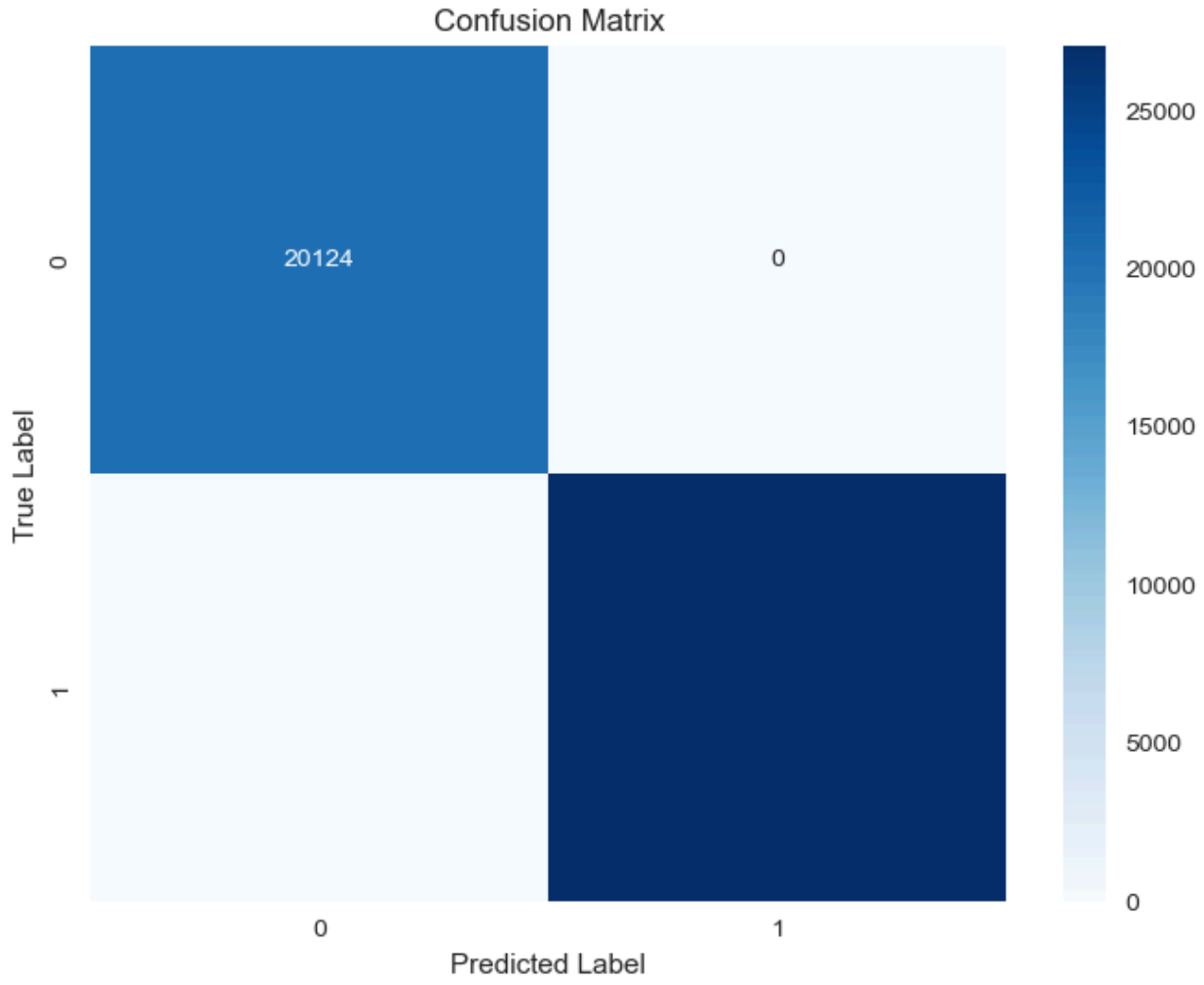


Figure 9. Detailed Classification Report

The Random Forest model achieves a high accuracy, precision, and recall when trained with the PhiUSIIL dataset.

6. Key Findings

This project's goal was to develop an effective phishing URL detection system using machine learning. There are several key insights that have emerged that contribute to understanding phishing URL characteristics and identifying effective detection methods.

6.1 Evaluation Metrics and Model Performance

To evaluate the performance of the models, we relied on a set of metrics like Accuracy, Precision, Recall (Sensitivity), F1 Score, and Matthews Correlation Coefficient (MCC). All these metrics tested the model's ability to differentiate between phishing and legitimate URLs across various dimensions and to ensure that the analysis covered both overall classification performance and the balance between false positives and false negatives. We tried and tested three primary algorithms - Random Forest, Decision Tree, and Logistic Regression. The Random Forest classifier was the best-performing model across all evaluation metrics. It had an Accuracy, Precision, Recall, and F1 Score of 1.0, and an Area Under the Receiver Operating Characteristic (ROC AUC) curve of 1.0. The Decision Tree classifier also achieved similarly high performance, with perfect metrics across the board. Logistic Regression had an Accuracy of 99.99% and slightly lower cross-validation scores compared to the tree-based models. This high performance is because the dataset is highly separable due to the presence of dominant features that distinguish phishing from legitimate URLs.

6.2 Insights from Model Performance

These observations indicate the presence of key URL characteristics that strongly differentiate phishing from legitimate URLs. These features are domain-specific patterns, suspicious keywords, and certain lexical characteristics often found in phishing URLs. Although

the models returned robust detection capabilities, the exceptionally high performance of tree-based classifiers, such as Random Forest and Decision Tree, raised concerns about potential overfitting. Perfect performance metrics, might sometimes mean that the models are overly tailored to the dataset. This overfitting can limit the model's generalization to new, unseen data, particularly when phishing strategies continue to evolve over time.

The ROC curves for all classifiers also confirmed their high performance. The perfect Random Forest scores indicate that the model consistently identifies phishing URLs without sacrificing its ability to recognize legitimate ones.

6.3 Limitations and Challenges

Despite the promising results, several limitations were identified during the project. One of the challenges was ensuring the model's generalizability. Although the PhiUSIIL dataset reflects real-world scenarios, the high performance of the models might be dataset-specific. Another limitation was the absence of external datasets for validation. Testing the model on independent datasets is essential to assess its robustness and adaptability to varying phishing strategies.

7. Recommendations

To address the limitations above, there are several steps that could be taken. First, the future work should focus on testing the models on external datasets to evaluate its generalizability and robustness. The test datasets should include URLs with up to date phishing patterns to ensure the model remains relevant in the dynamic cyber crime world. Also, exploring techniques to mitigate overfitting, such as pruning for tree-based models or applying

regularization techniques, could enhance the model's ability to generalize to unseen data. There should be progressive updates to the training dataset and incorporating new phishing URLs to help maintain the model's effectiveness over time. Other stakeholders such as developers, should integrate phishing detection algorithms into widely used tools such as browsers and email filters to enhance end-user protection. They can set the trained models as browser extensions where these tools can analyze URLs in real-time and provide immediate warnings to users when phishing threats are detected or as email filters to scan incoming emails for malicious links. It is also important to drop old ways of determining phishing urls like domain age, rank, or location, as they can lead to disruptions if the providers are unavailable and adopt self sustainable new ways that can guarantee efficiency.

Organizations need to equip their employees with the knowledge to identify and respond to phishing attempts using the best practices for example Regular Training Sessions where they could offer interactive workshops to educate employees about common phishing tactics, red flags in emails, and safe browsing practices or even organize phishing simulation exercises to assess and improve employees' ability to detect threats. They should also implement Advanced Detection Systems mostly as machine learning models integrated with organizational email servers to automatically flag or quarantine suspicious emails. In this case, they could combine URL analysis with contextual content evaluation to ensure comprehensive threat detection. Moreover, there is a need to not only set clear procedures for employees to report suspected phishing attacks but also ensure rapid containment and remediation measures are in place to minimize the impact of successful phishing attempts.

In conclusion, the policymakers can collaborate in combating phishing by establishing standardized frameworks and policies. Some ways they can achieve this is through enforcing a uniform reporting guideline that registers reports of phishing attacks to national cybersecurity agencies. In addition, they could record these urls in a centralized database that would provide

machine learning researchers and developers with access to up-to-date data for model training and testing. Finally, launching public awareness campaigns to sensitize citizens on recognizing phishing attempts and reporting them effectively would be a commendable step.

8. Conclusion

This research introduces PhiUSIIL, a robust dataset that contains 134,850 legitimate and 100,945 phishing URLs that was trained to detect phishing URLs using machine learning. This diverse dataset ensures improved model generalization and detection accuracy. In the statistical analysis, we revealed high accuracy and performance of tree-based algorithms such as Random Forest and Decision Tree. This calls for further testing on external datasets. In future, expanding the features beyond the urls could help capture deeper patterns and improve the detection capabilities of the model in return. Future research should focus on automated frameworks that can capture emerging phishing attack types in real-time. Again, the researchers should develop tools to eliminate phishing URLs across networks. More advancement of the model including large and up to date datasets will combat this prevalent cybercrime threat.

9. Bibliography

Adebowale, M., Lwin, K., Sánchez, E., & Hossain, M. (2019). Intelligent web-phishing detection and protection scheme using integrated features of images, frames and text. *Expert Systems with Applications, 115*, 300–313. http://dx.doi.org/10.1016/j.Eswa.2018.07.067.

Bose, I., & Leung, A. C. M. (2014). Do phishing alerts impact global corporations? A firm value analysis. *Decision Support Systems, 64*, 67–78. http://dx.doi.org/10.1016/j.dss.2014.04.006.

Chen, R. C., Dewi, C., Huang, S. W., & Caraka, R. E. (2020). Selecting critical features for data classification based on machine learning methods. *Journal of Big Data, 7*(1), 52.

Chiew, K. L., Yong, K. S. C., & Tan, C. L. (2018). A survey of phishing attacks: Their types, vectors and technical approaches. *Expert Systems with Applications, 106*, 1–20. http://dx.doi.org/10.1016/j.eswa.2018.03.050.

El Aassal, A., Baki, S., Das, A., & Verma, R. M. (2020). An in-depth benchmarking and evaluation of phishing detection research for security needs. *IEEE Access, 8*. http://dx.doi.org/10.1109/ACCESS.2020.2969780.

Ghiasi, M. M., & Zendehboudi, S. (2021). Application of decision tree-based ensemble learning in the classification of breast cancer. *Computers in Biology and Medicine, 128*, 104089.

Hall, M. A. (1999). *Correlation-based feature selection for machine learning* (PhD dissertation). The University of Waikato.

Kotsiantis, S. B., Kanellopoulos, D., & Pintelas, P. E. (2006). Data preprocessing for supervised learning. *International Journal of Computer Science, 1*(2), 111–117.

Krishnan, S., Franklin, M. J., Goldberg, K., & Wu, E. (2017). Boostclean: Automated error detection and repair for machine learning. *arXiv preprint arXiv:1711.01299.*

Li, Y., Yang, Z., Chen, X., Yuan, H., & Liu, W. (2019). A stacking model using URL and HTML features for phishing webpage detection. *Future Generation Computer Systems, 94*, 27–39. http://dx.doi.org/10.1016/j.future.2018.11.004.

Mohammad, R. M., Thabtah, F., & McCluskey, L. (2015a). Phishing website dataset: UCI machine learning repository. URL: https://archive.ics.uci.edu/ml/machine-learning-databases/00327/Training%20Dataset.arff. Accessed 28.10.2020.

Oest, A., Zhang, P., Wardman, B., Nunes, E., Burgis, J., Zand, A., Thomas, K., Doupé, A., & Ahn, G.-J. (2020). Sunrise to sunset: Analyzing the end-to-end life cycle and effectiveness of phishing attacks at scale. In *USENIX Security Symposium.*

Saarela, M., & Jauhiainen, S. (2021). Comparison of feature importance measures as explanations for classification models. *SN Applied Sciences, 3*(2), 272.

Sanchez-Paniagua, M., Fernandez, E. F., Alegre, E., Al-Nabki, W., & Gonzalez-Castro, V. (2022). Phishing URL detection: A real-case scenario through login URLs. *IEEE Access, 10*, 42949–42960. http://dx.doi.org/10.1109/ACCESS.2022.3168681.

Tan, C. L., Chiew, K. L., Musa, N., & Ibrahim, D. H. A. (2018). Identifying the most effective feature category in machine learning-based phishing website detection. *International Journal of Engineering & Technology, 7*(4.31), 1–6. http://dx.doi.org/10.14419/ijet.v7i4.31.23331.

Toloşi, L., & Lengauer, T. (2011). Classification with correlated features: Unreliability of feature ranking and solutions. *Bioinformatics, 27*(14), 1986–1994.

10. Appendix

```
# -*- coding: utf-8 -*-
"""Project Proposal and DataSet New1.ipynb

Automatically generated by Colab.

"""

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
from sklearn.cluster import KMeans
from sklearn.feature_selection import SelectKBest, f_classif
from scipy import stats
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score,
recall_score,
                             f1_score, confusion_matrix,
classification_report,
                             roc_curve, roc_auc_score)
from sklearn.pipeline import Pipeline

# Set styling for all plots
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")
plt.rcParams['figure.figsize'] = [12, 8]

df = pd.read_csv("/Users/Desktop/PhiUSIIL_Phishing_URL_Dataset.csv")

df.head()
```

```
print("A. Basic statistical summaries")
print("\nDataset shape:", df.shape)
print("\nColumn types:")
print(df.dtypes)
print("\nSummary statistics for numerical columns:")
print(df.describe())

print("\nClass distribution:")
print(df['label'].value_counts(normalize=True))

"""## 1. Class Distribution Analysis"""

def analyze_class_distribution(df):
   """Analyze and visualize the distribution of legitimate vs
phishing URLs"""
   plt.figure(figsize=(10, 6))
   class_dist = df['label'].value_counts()
   plt.pie(class_dist,
           labels=['Legitimate' if i == 1 else 'Phishing' for i in
class_dist.index],
           autopct='%1.1f%%',
           colors=['#2ecc71', '#e74c3c'])
   plt.title('Distribution of URLs in Dataset')
   plt.axis('equal')
   plt.show()

   print("\nClass Distribution Summary:")
   print(f"Total URLs: {len(df):,}")
   print(f"Legitimate URLs: {class_dist[1]:,}
({class_dist[1]/len(df)*100:.1f}%)")
   print(f"Phishing URLs: {class_dist[0]:,}
({class_dist[0]/len(df)*100:.1f}%)")

# Execute the analysis
analyze_class_distribution(df)

"""## 2. Numerical Features Analysis"""

def analyze_numerical_features(df):
```

```
    """Analyze key numerical features with box plots and statistical
tests"""
    key_features = ['URLLength', 'DomainLength', 'NoOfSubDomain',
                    'NoOfLettersInURL', 'NoOfDegitsInURL',
'NoOfOtherSpecialCharsInURL']

    # Create box plots for each feature
    fig, axes = plt.subplots(2, 3, figsize=(20, 12))
    axes = axes.ravel()

    for idx, feature in enumerate(key_features):
        sns.boxplot(x='label', y=feature, data=df, ax=axes[idx])
        axes[idx].set_xlabel('URL Type (0: Phishing, 1: Legitimate)')
        axes[idx].set_title(f'Distribution of {feature}')

        # Perform Mann-Whitney U test
        stat, p_value = stats.mannwhitneyu(
            df[df['label'] == 0][feature],
            df[df['label'] == 1][feature]
        )
        axes[idx].text(0.05, 0.95, f'p-value: {p_value:.2e}',
                       transform=axes[idx].transAxes)

    plt.tight_layout()
    plt.show()

# Execute the analysis
analyze_numerical_features(df)

"""## 3. Correlation Analysis"""

def create_correlation_matrix(df):
    """Create and visualize correlation matrix for important
features"""
    important_features = [
        'URLLength', 'DomainLength', 'URLSimilarityIndex',
'NoOfSubDomain',
        'NoOfLettersInURL', 'NoOfDegitsInURL', 'IsHTTPS',
'NoOfURLRedirect',
```

```
        'HasHiddenFields', 'HasPasswordField', 'label'
    ]

    corr_matrix = df[important_features].corr()

    plt.figure(figsize=(12, 10))
    mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
    sns.heatmap(corr_matrix, mask=mask, cmap='coolwarm', center=0,
                annot=True, fmt='.2f', square=True)
    plt.title('Correlation Matrix of Key Features')
    plt.tight_layout()
    plt.show()

    # Print strong correlations
    print("\nStrong correlations (|r| > 0.5):")
    for i in range(len(important_features)):
        for j in range(i+1, len(important_features)):
            if abs(corr_matrix.iloc[i,j]) > 0.5:
                print(f"{important_features[i]} -
{important_features[j]}: {corr_matrix.iloc[i,j]:.3f}")

# Execute the analysis
create_correlation_matrix(df)

"""## 4. Security Indicators Analysis"""

def analyze_security_indicators(df):
    """Analyze and visualize key security indicators"""
    security_features = ['IsHTTPS', 'HasPasswordField',
'HasHiddenFields',
                         'HasExternalFormSubmit', 'HasSocialNet']

    fig, axes = plt.subplots(2, 3, figsize=(20, 12))
    axes = axes.ravel()

    for idx, feature in enumerate(security_features):
        if idx < len(axes):
            # Calculate percentages for each category
            feature_dist =
```

```
df.groupby('label')[feature].value_counts(normalize=True).unstack() *
100

            # Create bar plot
            feature_dist.plot(kind='bar', ax=axes[idx])
            axes[idx].set_title(f'{feature} Usage by URL Type')
            axes[idx].set_xlabel('URL Type (0: Phishing, 1:
Legitimate)')
            axes[idx].set_ylabel('Percentage')
            axes[idx].legend(['Not Present (0)', 'Present (1)'])

            # Add chi-square test
            contingency = pd.crosstab(df['label'], df[feature])
            chi2, p_value = stats.chi2_contingency(contingency)[:2]
            axes[idx].text(0.05, 0.95, f'p-value: {p_value:.2e}',
                           transform=axes[idx].transAxes)

    plt.tight_layout()
    plt.show()

# Execute the analysis
analyze_security_indicators(df)

"""## 5. Feature Importance Analysis"""

def feature_importance_analysis(df):
    """Analyze feature importance using SelectKBest"""
    # Prepare features
    X = df.select_dtypes(include=[np.number]).drop('label', axis=1)
    y = df['label']

    # Scale features
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Select top features
    selector = SelectKBest(score_func=f_classif, k=10)
    selector.fit(X_scaled, y)
```

```
    # Get feature scores
    feature_scores = pd.DataFrame({
        'Feature': X.columns,
        'Score': selector.scores_
    }).sort_values('Score', ascending=False)

    # Plot feature importance
    plt.figure(figsize=(12, 6))
    sns.barplot(x='Score', y='Feature', data=feature_scores.head(10))
    plt.title('Top 10 Most Important Features')
    plt.xlabel('F-Score')
    plt.tight_layout()
    plt.show()

    return feature_scores

# Execute the analysis
feature_scores = feature_importance_analysis(df)
print("\nTop 10 Most Important Features:")
print(feature_scores.head(10))

numeric_cols = df.select_dtypes(include=[np.number]).columns
plt.figure(figsize=(12, 10))
sns.heatmap(df[numeric_cols].corr(), cmap='coolwarm', annot=False)
plt.title('Correlation Heatmap of Numeric Features')
plt.show()

# Separate features and target
# Remove non-numeric and irrelevant columns
X = df.select_dtypes(include=[np.number]).drop('label', axis=1)
y = df['label']

"""## 6. Feature Importance Across Multiple Algorithms"""

def compare_feature_importance(X, y):
    """
    Compare feature importance across different algorithms
    """
    # Initialize classifiers
```

```
    classifiers = {
        'Random Forest': RandomForestClassifier(n_estimators=100,
random_state=42),
        'Decision Tree': DecisionTreeClassifier(random_state=42),
        'Logistic Regression': LogisticRegression(random_state=42,
max_iter=1000)
    }

    # Store feature importances
    feature_importances = {}

    # Prepare a figure for visualization
    plt.figure(figsize=(15, 10))

    # Iterate through classifiers
    for idx, (name, clf) in enumerate(classifiers.items(), 1):
        # Prepare pipeline with scaling
        pipeline = Pipeline([
            ('scaler', StandardScaler()),
            ('classifier', clf)
        ])

        # Fit the pipeline
        pipeline.fit(X, y)

        # Extract classifier
        actual_clf = pipeline.named_steps['classifier']

        # Get feature importances based on algorithm type
        if name == 'Random Forest':
            importances = actual_clf.feature_importances_
        elif name == 'Decision Tree':
            importances = actual_clf.feature_importances_
        elif name == 'Logistic Regression':
            importances = np.abs(actual_clf.coef_[0])

        # Create a DataFrame of feature importances
        feature_imp_df = pd.DataFrame({
            'feature': X.columns,
```

```
            'importance': importances
        }).sort_values('importance', ascending=False)

        # Store in dictionary
        feature_importances[name] = feature_imp_df

        # Subplot for each classifier
        plt.subplot(2, 2, idx)
        sns.barplot(x='importance', y='feature',
                    data=feature_imp_df.head(10),
                    palette='viridis')
        plt.title(f'{name} - Top 10 Features')
        plt.xlabel('Feature Importance')
        plt.ylabel('Features')

    plt.tight_layout()
    plt.show()

    return feature_importances

# Run feature importance comparison
feature_importances = compare_feature_importance(X, y)

"""## 7. Model Performance Comparison"""

def compare_model_performance(X, y):
    """
    Compare performance of different classifiers
    """
    # Split the data
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    # Initialize classifiers
    classifiers = {
        'Random Forest': RandomForestClassifier(n_estimators=100,
random_state=42),
        'Decision Tree': DecisionTreeClassifier(random_state=42),
```

```
        'Logistic Regression': LogisticRegression(random_state=42,
max_iter=1000)
    }

    # Performance metrics storage
    performance_metrics = {}

    # Prepare a figure for ROC curves
    plt.figure(figsize=(10, 8))

    # Iterate through classifiers
    for name, clf in classifiers.items():
        # Prepare pipeline with scaling
        pipeline = Pipeline([
            ('scaler', StandardScaler()),
            ('classifier', clf)
        ])

        # Perform cross-validation
        cv_scores = cross_val_score(pipeline, X, y, cv=5)

        # Fit the pipeline
        pipeline.fit(X_train, y_train)

        # Predict
        y_pred = pipeline.predict(X_test)
        y_pred_proba = pipeline.predict_proba(X_test)[:, 1]

        # Calculate performance metrics
        metrics = {
            'Accuracy': accuracy_score(y_test, y_pred),
            'Precision': precision_score(y_test, y_pred),
            'Recall': recall_score(y_test, y_pred),
            'F1 Score': f1_score(y_test, y_pred),
            'ROC AUC': roc_auc_score(y_test, y_pred_proba),
            'Cross-Validation Mean': cv_scores.mean(),
            'Cross-Validation Std': cv_scores.std()
        }
```

```
        performance_metrics[name] = metrics

        # Plot ROC curve
        fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
        plt.plot(fpr, tpr, label=f'{name} (AUC = {metrics["ROC
AUC"]:.2f})')

    # ROC curve details
    plt.plot([0, 1], [0, 1], linestyle='--', label='Random
Classifier')
    plt.title('Receiver Operating Characteristic (ROC) Curve')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.legend()
    plt.show()

    # Print performance metrics
    print("Model Performance Metrics:")
    for name, metrics in performance_metrics.items():
        print(f"\n{name} Performance:")
        for metric, value in metrics.items():
            print(f"{metric}: {value:.4f}")

    return performance_metrics

# Run model performance comparison
performance_metrics = compare_model_performance(X, y)

"""## 8. Detailed Classification Report"""

def generate_detailed_classification_report(X, y):
    """
    Generate detailed classification report for Random Forest
    """
    # Split the data
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
```

```
    # Prepare pipeline
    pipeline = Pipeline([
        ('scaler', StandardScaler()),
        ('classifier', RandomForestClassifier(n_estimators=100,
random_state=42))
    ])

    # Fit the pipeline
    pipeline.fit(X_train, y_train)

    # Predict
    y_pred = pipeline.predict(X_test)

    # Generate and print classification report
    print("\nDetailed Classification Report:")
    print(classification_report(y_test, y_pred))

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title('Confusion Matrix')
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.show()

# Generate detailed classification report
generate_detailed_classification_report(X, y)

"""## 9. Summary Statistics by URL Type"""

# Calculate summary statistics for key numerical features by URL type
key_features = ['URLLength', 'DomainLength', 'NoOfSubDomain',
'NoOfLettersInURL',
                'NoOfDegitsInURL', 'NoOfOtherSpecialCharsInURL']

print("Summary Statistics for Legitimate URLs (label=1):")
print(df[df['label'] == 1][key_features].describe())
print("\nSummary Statistics for Phishing URLs (label=0):")
```

```
print(df[df['label'] == 0][key_features].describe())

"""## 10. HTTPS usage in phishing vs legitimate URLs"""

https_usage = df.groupby('label')['IsHTTPS'].mean()
plt.figure(figsize=(8, 6))
https_usage.plot(kind='bar')
plt.title('HTTPS Usage in Phishing vs Legitimate URLs')
plt.xlabel('Label (0: Phishing, 1: Legitimate)')
plt.ylabel('Proportion of HTTPS Usage')
plt.show()

"""## 11. Potential relationships between variables"""

# Relationship between URL length and legitimacy
plt.figure(figsize=(10, 6))
sns.boxplot(x='label', y='URLLength', data=df)
plt.title('URL Length vs Legitimacy')
plt.show()

# Relationship between TLD length and legitimacy
plt.figure(figsize=(10, 6))
sns.boxplot(x='label', y='TLDLength', data=df)
plt.title('TLD Length vs Legitimacy')
plt.show()

print("\nTop 10 most common TLDs:")
print(df['TLD'].value_counts().head(10))

print("\nMean values of key features for phishing and legitimate
URLs:")
key_features = ['URLLength', 'DomainLength', 'NoOfSubDomain',
'TLDLength', 'NoOfURLRedirect', 'HasHiddenFields']
print(df.groupby('label')[key_features].mean())

# Separate features and target
X = df.drop(['label', 'FILENAME', 'URL', 'Domain', 'TLD', 'Title'],
axis=1)
y = df['label']
```

```
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Question 1: Which features are the most significant indicators
of a phishing URL?")

# Train a Random Forest classifier
rf_classifier = RandomForestClassifier(n_estimators=100,
random_state=42)
rf_classifier.fit(X_train_scaled, y_train)

# Get feature importances
feature_importance = rf_classifier.feature_importances_
feature_names = X.columns

# Sort features by importance
sorted_idx = np.argsort(feature_importance)
sorted_features = feature_names[sorted_idx]
sorted_importance = feature_importance[sorted_idx]

# Plot feature importances
plt.figure(figsize=(12, 8))
plt.barh(range(len(sorted_importance)), sorted_importance)
plt.yticks(range(len(sorted_importance)), sorted_features)
plt.xlabel('Feature Importance')
plt.title('Random Forest Feature Importance for Phishing URL
Detection')
plt.tight_layout()
plt.show()

print("\nTop 10 most important features:")
for i in range(1, 11):
```

```
    print(f"{i}. {sorted_features[-i]}: {sorted_importance[-i]:.4f}")

# Define URL-based and webpage content-based features
url_features = ['URLLength', 'DomainLength', 'IsDomainIP',
'URLSimilarityIndex', 'CharContinuationRate',
                'TLDLegitimateProb', 'URLCharProb', 'TLDLength',
'NoOfSubDomain', 'HasObfuscation',
                'NoOfObfuscatedChar', 'ObfuscationRatio',
'NoOfLettersInURL', 'LetterRatioInURL',
                'NoOfDegitsInURL', 'DegitRatioInURL',
'NoOfEqualsInURL', 'NoOfQMarkInURL',
                'NoOfAmpersandInURL', 'NoOfOtherSpecialCharsInURL',
'SpacialCharRatioInURL', 'IsHTTPS']

webpage_features = ['LineOfCode', 'LargestLineLength', 'HasTitle',
'DomainTitleMatchScore',
                    'URLTitleMatchScore', 'HasFavicon', 'Robots',
'IsResponsive', 'NoOfURLRedirect',
                    'NoOfSelfRedirect', 'HasDescription', 'NoOfPopup',
'NoOfiFrame', 'HasExternalFormSubmit',
                    'HasSocialNet', 'HasSubmitButton',
'HasHiddenFields', 'HasPasswordField', 'Bank',
                    'Pay', 'Crypto', 'HasCopyrightInfo', 'NoOfImage',
'NoOfCSS', 'NoOfJS', 'NoOfSelfRef',
                    'NoOfEmptyRef', 'NoOfExternalRef']

# Function to train and evaluate a model
def train_and_evaluate(X_train, X_test, y_train, y_test,
feature_set_name):
    X_train_set = X_train[feature_set_name]
    X_test_set = X_test[feature_set_name]

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train_set)
    X_test_scaled = scaler.transform(X_test_set)

    rf_classifier = RandomForestClassifier(n_estimators=100,
random_state=42)
    rf_classifier.fit(X_train_scaled, y_train)
```

```
    y_pred = rf_classifier.predict(X_test_scaled)
    accuracy = accuracy_score(y_test, y_pred)

    print(f"\nPerformance using {feature_set_name} features:")
    print(f"Accuracy: {accuracy:.4f}")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

# Evaluate models using URL-based and webpage content-based features
train_and_evaluate(X_train, X_test, y_train, y_test, url_features)
train_and_evaluate(X_train, X_test, y_train, y_test,
webpage_features)

# Evaluate model using all features
all_features = url_features + webpage_features
train_and_evaluate(X_train, X_test, y_train, y_test, all_features)
```